Names: Ishimwe Pacis Hanyurwimfura
Student Number: 3787234

Link to Excel with results running time for different processes, modes, function

Link to Github Repository

Assignment 3 Report

A. Design Summary

a. Adaptive Integration Method

The program computes the integral on the interval [0,1] using Adaptive Simpson's Rule. For each interval [a,b], it first computes one Simpson estimate on the whole interval then another estimate after splitting the interval at the midpoint. The difference between these two estimates is used to approximate the error and if the error is small enough, the interval is accepted. Otherwise, it is split again recursively. To avoid endless recursion, the program uses a safety limit MAX_DEPTH = 50. This same adaptive method is used in all three modes.

b. Reused Serial and MPI Dynamic Design

Since Assignment 3 builds on Assignment 2, I reused the serial implementation and the MPI dynamic master/worker version. In Mode 0, one process computes the full integral and serves as the correctness and timing baseline. In Mode 1, rank 0 is the master and the other ranks are workers. The master keeps a global pool of tasks, and workers repeatedly ask for work, process intervals, and send back newly created sub-intervals when needed. Each task stores the interval bounds, tolerance, and recursion depth in the form (a,b,tol,depth). I kept the same idea of using K=4 * the number of workers so that enough tasks are available early without creating too much communication overhead.

c. MPI Dynamic Communication and Termination

The dynamic MPI mode uses tag-based communication between the master and workers. A worker sends TAG_READY when it is free, the master sends TAG_TASK when it assigns work, a worker sends TAG_NEW_TASK when a split creates another task, TAG_STOP is used to end execution, and TAG_REPORT is used to return the final subtotal and interval count. Workers use a local stack to process tasks which lets them keep one half of a split interval and return the other half to the master. This helps with dynamic load balancing. The master only stops the workers when the global task pool is empty and no worker is still active, which prevents stopping too early.

d. Hybrid MPI Design

Here rank 0 acts only as the coordinator, while the worker ranks receive coarse intervals statically, and after that, each worker processes all of its assigned work locally. If an interval needs to be split, both child intervals stay on the same worker. They are not sent back to the master which is the main difference from the dynamic MPI

e. OpenMP Strategy

Inside each hybrid worker, I used OpenMP to parallelize the local recursive work. The worker starts an OpenMP parallel region, and one thread creates tasks for the assigned coarse intervals. The recursive hybrid function can also create more OpenMP tasks, but only up to a small depth limit so that the program does not create too many very small tasks and atomic updates are used when adding to the local subtotal and accepted interval count. So I used MPI to divide work across processes, while OpenMP was used to speed up the local recursive computation inside each worker.

B. Results Tables

The whole results can be found in this Google sheets all the 10 runs per each mode function and processor being (2,4,8) and removing the first run .
P.S My laptop couldn't manage over 8 processes.

Below are the runtime summary :

Function 1, tolerance = 1e-6

Runtime summary

Processes	Mode 0 Avg Runtime (s)	Mode 1 Avg Runtime (s)	Mode 2 Best Avg Runtime (s)	Best Hybrid Threads
2	0.000010	0.001877	0.000058	1
4	0.000010	0.002943	0.000074	1
6	0.000010	0.002160	0.000079	1
8	0.000010	0.000933	0.000077	1

SpeedUp & Efficiency

Processes	Mode 1 Speedup	Mode 2 Best Speedup	Best P,T (Process, thread)
2	0.005209	0.167300	2,1
4	0.003323	0.131540	4,1
6	0.004527	0.123769	6,1
8	0.010476	0.126984	8,1

Processes	Mode 1 Efficiency	Mode 2 Best Efficiency	Best P,T (Process, thread)
2	0.002604	0.083650	2,1
4	0.000831	0.032885	4,1
6	0.000754	0.020628	6,1
8	0.001310	0.015873	8,1

Function 1, tolerance = 1e-8

Runtime Summary

Processes	Mode 0 Avg Runtime (s)	Mode 1 Avg Runtime (s)	Mode 2 Best Avg Runtime (s)	Best Hybrid Threads
2	0.000018	0.002526	0.000088	1
4	0.000018	0.004516	0.000099	1
6	0.000018	0.002457	0.000097	1
8	0.000018	0.004240	0.000098	1

Speedup & Efficiency

Processes	Mode 1 Speedup	Mode 2 Best Speedup	Best P,T (Process, thread)
2	0.007302	0.209596	2,1
4	0.004084	0.186727	4,1
6	0.007506	0.189498	6,1
8	0.004351	0.189066	8,1

Processes	Mode 1 Efficiency	Mode 2 Best Efficiency	Best P,T (Process, thread)
2	0.003651	0.104798	2,1
4	0.001021	0.046682	4,1
6	0.001251	0.031583	6,1
8	0.000544	0.023633	8,1

Function 2, tolerance = 1e-8

Runtime summary

Processes	Mode 0 Avg Runtime (s)	Mode 1 Avg Runtime (s)	Mode 2 Best Avg Runtime (s)	Best Hybrid Threads
2	0.000941	0.007942	0.000857	2
4	0.000941	0.004869	0.000515	1

6	0.000941	0.004783	0.000411	1
8	0.000941	0.004449	0.000376	1

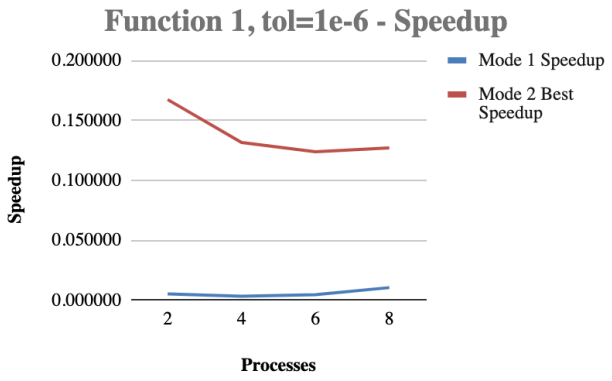
Speedup & Efficiency

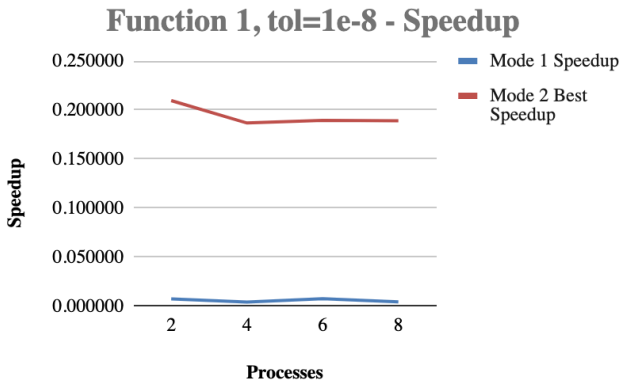
Processes	Mode 1 Speedup	Mode 2 Best Speedup	Best P,T (Process, thread)
2	0.118465	1.097744	2,2
4	0.193232	1.826575	4,1
6	0.196724	2.288649	6,1
8	0.211489	2.503104	8,1

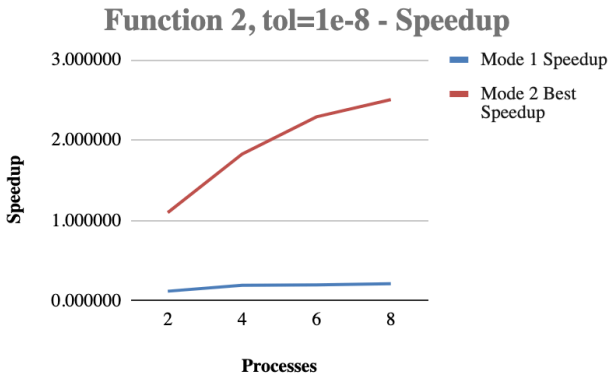
Processes	Mode 1 Efficiency	Mode 2 Best Efficiency	Best P,T (Process, thread)
2	0.059233	0.274436	2,2
4	0.048308	0.456644	4,1
6	0.032787	0.381441	6,1
8	0.026436	0.312888	8,1

C. Plots

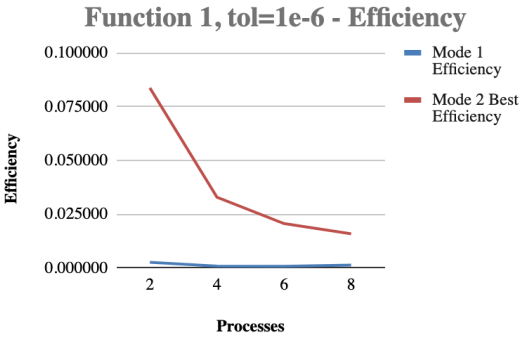
a. SpeedUp vs Number of Processes

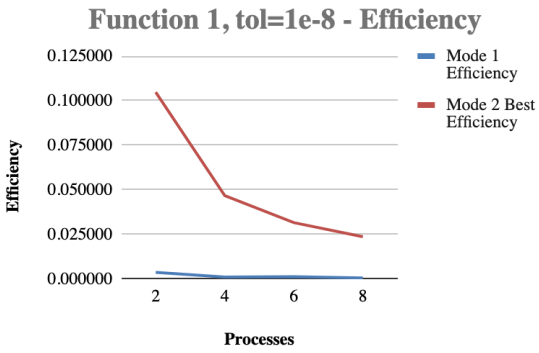


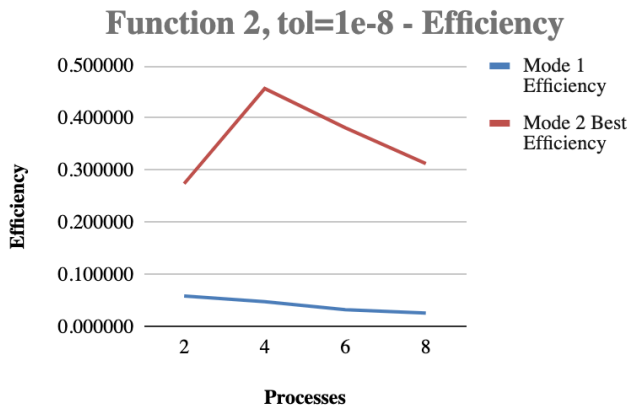




b. Efficiency Vs Number of Processes







D. Discussion

a. Why does MPI dynamic perform well for irregular workloads?
ANS/ MPI dynamic perform well for irregular workloads because workers do not stay stuck with one fixed part of the interval. When a worker becomes free, it asks for more work, and newly created sub-intervals can be sent back to the master and redistributed. This helps balance uneven adaptive work better than a fixed partition.

b. Why might hybrid perform worse?

ANS/Hybrid can perform worse because once the master gives out the coarse intervals, work is no longer redistributed between processes. If one worker gets a heavier part of the domain, the others cannot help. It can also lose time to OpenMP overhead when the local work is too small. We can see this like for Function 1, where the best hybrid runtimes are still much larger than the serial baseline when the thread is 1 and increasing threads often makes runtime worse instead of better.

c. Does OpenMP compensate for lack of MPI load balancing?
ANS/ Only partly. OpenMP helps each worker process its own local recursion faster but it does not fix imbalance between different MPI workers. In the results we can see the best hybrid setting is often T=1 for Function 1 and also for most Function 2 cases, which means adding more threads did not fully compensate for

the missing global redistribution, the only case where extra threads helped was Function 2 at 1e-8 with P=2, where hybrid with T=2 was slightly faster than T=1.

d. When does hybrid outperform MPI?
ANS/ Hybrid outperforms MPI in my results and significantly when the workload is large enough for local computation to dominate communication overhead. For Function 2, hybrid is much faster than MPI at both tolerances for valid configurations. For example, at Function 2 and 1e-8, mode 1 at P=8 averages about 0.00445 s, while the best hybrid at P=8 is about 0.000376 s , but when the work is very small like For Function 1, even though hybrid appears faster than mode 1 both are still slower than serial.

e. What is the impact of thread count and process count?
ANS/ Increasing the number of processes helped the hybrid mode more than increasing the number of threads. For example, with Function 2 at 1e-8, the best hybrid runtime improved from about 0.000857 s at P=2 to about 0.000376 s at P=8. But increasing threads often added overhead, in my experiments, process count had the stronger positive effect while thread count only helped in a few heavier cases, we can see that most best hybrid runtime was with 1 thread.